

AWS Certified Generative AI Developer — Professional

The Use Case

- Large North American healthcare provider — medical document analysis feeding an EHR system
- **Document types:** clinical notes, lab reports, radiology findings, patient histories
- **Constraints:** HIPAA compliance, 99.9% availability, high accuracy in medical terminology, must adapt to evolving medical practice
- **Scale:** 50,000 documents per day

Task 1.1 vs Task 1.2 — The Core Contrast

	Task 1.1 (Bank)	Task 1.2 (Healthcare)
How the model is improved	RAG — documents injected into the prompt	Fine-tuning — the model itself is changed
Why	Facts change frequently	Medical terminology and output format must be learned
Number of models	One (Claude)	Two (Claude + Titan) plus fallbacks

1. Model Selection

Step 1 — Define criteria first

Written **before** testing, otherwise results get rationalised to fit a preferred model.

Metric	Threshold	Weight	
Medical accuracy	> 90%	40%	(HIPAA — non-negotiable)
PHI detection	> 99%	30%	
JSON validity	> 95%	15%	
Latency p95	< 3s	10%	
Cost/doc	< \$0.02	5%	

Step 1b — Capability analysis comes before benchmarking

If a model lacks the required **capability**, its accuracy score is irrelevant. Check capability fit first.

Capability	Why it can disqualify a model
Context window size	50-page clinical documents won't fit a small window, however accurate the model is
Token limits	Separate caps on input and output
Multimodal support	Scanned documents require it — a text-only model cannot process them at all
Specialised abilities	Code generation, mathematical reasoning — models differ sharply

Bedrock benchmark dimensions: reasoning (multi-step thinking), knowledge (domain depth), safety (toxicity and harmful output), and multilingual coverage.

Step 2 — Prepare the dataset

- JSONL file in S3: `prompt` + `referenceResponse` pairs
- 100–300 examples is enough, but they must be **representative** — include rare diseases and poor-quality OCR, not just clean cases
- **HIPAA:** de-identify, stay within a BAA, encrypt with KMS

Step 3 — Bedrock Model Evaluation

Type	Characteristics
Automatic	Built-in metrics (accuracy, robustness, toxicity). Fast and cheap. Good for shortlisting.
LLM-as-a-judge	One model scores another. Supports custom metrics — more useful in medicine, since built-in metrics don't understand clinical correctness.
Human	Own work team (clinicians) or AWS-managed. Slow and expensive, but required for final clinical sign-off.

In a medical context all three are used: automatic to shortlist, LLM-judge for depth, human for sign-off.

Step 4 — Latency and cost are measured separately

- **Model Evaluation does not give you these.** Run a separate load script and read CloudWatch.
- Metrics: `InvocationLatency` (p50/p95/p99), `InputTokenCount`, `OutputTokenCount`

- Cost = tokens × price × 50,000 docs/day. **This is often where the decision flips** — the most accurate model may be 5× the price.

Total Cost of Ownership is more than inference price:

- Inference cost per token
- **Integration complexity** — how much code and engineering time to adopt it
- **Ongoing maintenance** — monitoring and tuning effort
- Fine-tuning cost, if required
- Supporting infrastructure (vector store, endpoints)

A cheaper model that takes three months to integrate is not cheaper.

Step 5 — Limitation analysis

Four methods, each catching something different:

Method	How it works
Slicing	Tag the dataset by category, then break results down by slice. Automatic — but the tagging and the choice of slices are manual.
Reading failures	Manually read 30-50 failed cases. Patterns emerge that no metric reports. Requires a clinician in this domain.
Adversarial testing	Deliberately break it: negation ("denies chest pain"), abbreviations, contradictions, prompt injection embedded in documents.
Production drift	Ongoing, via SageMaker Model Monitor. Not all limitations are findable before launch.

```
common_disease: 96%
rare_disease:   71%  <- hidden inside an 89% average
handwritten:    64%
```

Model Evaluation gives you a score. It does not give you the limitation — you have to go looking for it.

Document each limitation with its mitigation: hallucination → Guardrails grounding check; format drift → schema validation; knowledge cutoff → RAG for current guidelines. Record in a **Model Card**.

Step 6 — Two models, not one

```
Claude → medical understanding
Titan  → structured extraction
```

Different jobs need different capabilities. Using the largest model for every task wastes money. This "right model for the right job" pattern appears in exam scenarios.

Store the choice in AppConfig, never hardcoded.

Step 7 — Document the selection rationale

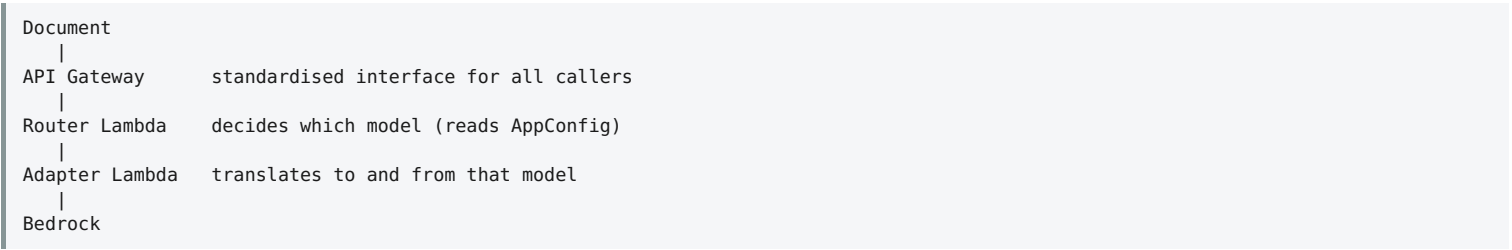
- Record both the **quantitative benchmarks** and the **qualitative assessment** behind the decision
- **Why:** in six months a new model appears. Without the recorded rationale you cannot make a like-for-like comparison — and in a regulated industry, the regulator may ask why this model was chosen
- Captured in a **Model Card**

2. Abstraction Layer

The problem

Claude and Titan have different request and response shapes. Calling Bedrock directly scatters `if model == ...` branches across the codebase. Adding a model then means editing twenty places — the cost of switching becomes so high you never switch. That is vendor lock-in.

The three layers



API Gateway

- The EHR always sends the same request shape and always gets the same response shape
- **The EHR never knows which model ran.** Swapping models has zero impact on it.

Feature	What it does
Custom authorizers	A Lambda that authorises each request — business logic beyond Cognito (which department, which document types this caller may submit)
Request/response transformations	Mapping templates reshape the payload before it reaches Lambda and on the way back — simple normalisation without writing code

Router Lambda

- Decides on two inputs: **what the task is**, and **what AppConfig says** for that task
- The second matters more — changing the AppConfig value sends the same question to a different model, with no code change

Adapter Lambda — three jobs, not one

1. **Translate inbound** — one internal request shape becomes Claude's `messages` / `max_tokens` or Titan's `inputText` / `textGenerationConfig`
2. **Call Bedrock** — the smallest part of the job
3. **Translate outbound** — Claude's `content[0].text` or Titan's `results[0].outputText` both become one normalised response

Also handles: retry with exponential backoff, timeouts, JSON schema validation, model-specific prompt formatting.

The adapter is the only place model-specific detail exists. A new model means one new adapter — nothing else changes.

AWS AppConfig

AppConfig is a settings file, not a place. The Router and Adapter are not "inside" it — the Router simply reads a value from it. Requests never travel "into" AppConfig.

What	Where it lives
Decision logic	Router Lambda (code)
Model-specific translation	Adapter Lambda (code)
Bedrock invocation	Inside the Adapter Lambda
The setting only	AppConfig

Why not environment variables? Changing those requires redeploying the Lambda. AppConfig changes at runtime while the function keeps running.

Feature flags for A/B testing:

```
{ "new_model_rollout": {
  "enabled": true,
  "percentage": 10,
  "departments": ["radiology"] } }
```

- New model runs on 10% of documents, radiology only. Then 50%, then 100%.
- Bad result → set to 0% instantly. Config change, not a deployment.
- AppConfig deployment strategies: linear, canary, all-at-once — with **automatic rollback** on a CloudWatch alarm

3. Resilience

Three kinds of failure, three different remedies. This distinction is the concept being tested.

Failure	Meaning	Remedy
Transient	One-off throttle or network glitch	Retry with exponential backoff
Sustained	Model or region genuinely down	Circuit breaker, cross-region
Degraded	Running, no errors, but output quality has dropped	CloudWatch custom metrics

Retry with exponential backoff

- Wait 1s, 2s, 4s between attempts — doubling gives a throttled service room to recover
- **Jitter** adds randomness so hundreds of Lambdas don't retry in lockstep
- Implemented inside the Adapter Lambda

Circuit breaker — AWS Step Functions

Retrying is pointless when a model is fully down; it just burns time. A circuit breaker is a fuse — its job is to *stop trying*.

State	Behaviour
Closed	Normal — requests flow through
Open	Too many failures → stop attempting entirely , go straight to fallback
Half-open	After a cooldown, send one test request. Success → back to Closed

Step Functions holds this state and defines the retry logic, error catching and alternative paths.

Cross-region failover

- **Bedrock cross-region inference profiles** — Bedrock distributes inference across regions
- **Route 53 health checks** — traffic directed away from unhealthy endpoints
- Transcript claim: a regional outage caused only a **3-minute** interruption

Graceful degradation

"Nothing" is the wrong answer. "Less" is the right one.

```
Claude (best)
| fail
Smaller / cheaper model
| fail
Rule-based extraction (regex, keyword lists)
| fail
Human review queue
```

Each step is worse but still functioning — like a hospital generator keeping ICU and emergency running rather than shutting the building down.

Multi-model ensemble

- The inverse case: send complex documents to **two** models and compare
- Agreement → high confidence. Disagreement → route to human review
- This is about **confidence, not resilience**. Expensive, so reserved for difficult documents.

Catching degraded performance — CloudWatch

This is the sneakiest failure: **latency is fine, error rate is zero, and the answers are wrong**. Standard monitoring cannot see it.

- Custom metrics needed: extraction confidence, JSON schema failure rate, sudden rise in documents returning zero diagnoses
- Alarm → **automated remediation**: switch model via AppConfig, or shift traffic to another region

4. Fine-tuning and LoRA

RAG vs fine-tuning — the exam's favourite distinction

	RAG	Fine-tuning
Supplies	New facts	New behaviour
Use when	Facts change frequently	Language, style or output format must be learned
Updating	Update the documents	Retrain

Rule: new *knowledge* → RAG. New *way of doing things* → fine-tuning.

Fine-tuning does not fix the knowledge cutoff. It teaches format and style, not fresh facts. Real systems use both: format from the fine-tuned model, facts from retrieval.

LoRA — Low-Rank Adaptation

Full fine-tuning adjusts billions of weights: expensive GPU time, a whole new model to store, and a separate full model per use case.

LoRA freezes the base model and trains a **small added layer** instead — roughly 1% the size.

- Cheaper and faster to train
- Only the adapter is stored, not a full model copy
- **One base model, many adapters** — radiology and pathology adapters over the same base
- Rollback is removing the adapter; the base is untouched
- **Trade-off:** slightly below full fine-tuning on accuracy — roughly 90% of the benefit at 5% of the cost

Where to run it

Option	Characteristics
Bedrock Custom Models	Simple — supply data, Bedrock trains. Limited model support, less control.
SageMaker AI	Full control — own training script, own hyperparameters, any model. Used in this scenario.

Steps: configure SageMaker training environment → configure LoRA parameters → execute fine-tuning job → evaluate performance.

LoRA parameters: `rank (r)` — adapter size, smaller is cheaper but learns less; `alpha` — strength of influence; `target_modules` — which parts of the model it attaches to.

Data preparation

- All document types — clinical notes, lab, radiology, history
- Easy and hard cases, not just clean ones
- **De-identified** — PHI cannot enter training data under HIPAA
- Separate train / validation / test splits

Roughly 80% of fine-tuning effort is data preparation. Launching the training job is one command.

5. Model Registry and Lifecycle

SageMaker Model Registry

Function	Purpose
Versioning	Every model gets a version. Rollback = point at an earlier one.
Lineage	Which dataset, which code, which parameters produced this model. Required when a regulator asks, months later, how a given decision was made.
Approval workflow	Status sits at <code>PendingManualApproval</code> . A clinician reviews results and approves before the pipeline proceeds.

The clinical validation gate is **deliberately manual**. Automating the pipeline does not mean removing the human.

CodePipeline + CodeBuild

- **CodePipeline** orchestrates the stages; **CodeBuild** runs the tests and builds
- Healthcare-specific stages: **medical accuracy test** → **HIPAA compliance check (PHI leakage)** → **EHR integration test**
- These are domain gates, not ordinary software tests

Blue/Green deployment — Lambda aliases

Blue = current model (serving traffic) Green = new model (ready, no traffic)

- Switch traffic at once; bad result → back to Blue in seconds
- A Lambda **alias** is a name pointing at a version: `prod` → `version 7`. Deploy = repoint to 8; rollback = repoint to 7.
- **Weighted aliases** allow gradual shift: `prod`: 90% → v7, 10% → v8

Strategy	How traffic moves
Blue/Green	All at once — 0% or 100%. Fastest rollback, but full exposure at cutover.
Canary	Incrementally — 5% → 25% → 50% → 100%, via weighted aliases. A problem at 5% affects only 5% of documents.

Canary is generally the safer choice in healthcare for exactly that reason.

What it switches	
AppConfig feature flag	A config value — which model to choose
Lambda alias	A code version — which Lambda runs

SageMaker Model Monitor — after deployment

A deployed model degrades over time even though the code never changed — because the world changes around it. This is drift.
--

- In healthcare: new terminology, new EHR document templates, new conditions and codes
- Model Monitor compares live data against a baseline and alerts when divergence grows
- Four monitoring types: **data quality**, **model quality**, **bias drift**, **feature attribution drift**

Service	Question it answers
CloudWatch	Is the system running?
SageMaker Model Monitor	Are the answers still correct?

Retirement and knowledge transfer

- Preserve the retired model's performance record and known failure modes
- Test the replacement against the **same baseline**
- Without this, each new model repeats the previous one's mistakes and nobody remembers what was already learned

The Full Lifecycle

```
Fine-tuned model
-> Model Registry (version + lineage)
-> Clinical validation (human approval)
-> CodePipeline
    CodeBuild: accuracy / HIPAA / EHR tests
-> Blue/Green deploy (Lambda alias)
-> Model Monitor (watches for drift)
-> drift detected -> retrain
-> retire old model, transfer knowledge
```

This is a loop, not a straight line. Monitoring feeds back into training.

Reported Outcomes

- 50,000 documents/day, 99.97% availability, 94% accuracy in medical entity extraction
- Regional outage handled with a 3-minute interruption
- 72% reduction in manual document review; 58% improvement in structured data completeness in the EHR

Service limits and feature availability change. Verify specific numbers against current AWS documentation before the exam.